Indian Institute of Technology Bhilai

Group Assignment 2

From Attention to Transformers

Translation, Rating Prediction, Summarization and Story Generation

Course: DSL 504 – Natural Language Processing

Technical Report (Questions 1–4)

Submitted By

Shashank Yadav	12342010
Lakshay Gupta	12341300
Rathod Chetan Kumar A.	12341750
Kabeer More	12341030
Rohit Raghuvanshi	12341820

Code and notebooks:
https://github.com/iamshashankyadav/NLP_GRA_2

For any queries, please contact Lakshay Gupta
lakshayg@iitbhilai.ac.in | +91 99111 18016

October 2026

Contents

1 Objective and Overview **2**

2 Team and Equal Contribution **2**

3 Q1 – Attention-Based Machine Translation (English to Hindi) **4**
3.1 Goal and setup . . . 4
3.2 Why the baseline struggles . . . 4
3.3 Training comparison . . . 5
3.4 What the model looks at . . . 6
3.5 Error analysis and length . . . 6

4 Q2 – RoBERTa vs BiGRU for Review Rating Prediction **7**
4.1 Setup . . . 7
4.2 Results . . . 7
4.3 Ordinal behaviour: how wrong are the mistakes? . . . 8
4.4 Ablation, qualitative cases and conclusions . . . 8

5 Q3 – Abstractive Summarization with a From-Scratch Transformer **9**
5.1 Problem and approach . . . 9
5.2 Data preparation . . . 10
5.3 Model architecture . . . 11
5.4 Training procedure . . . 13
5.5 Decoding . . . 14
5.6 Evaluation protocol . . . 14
5.7 Attention visualisation and qualitative analysis . . . 15
5.8 Results . . . 15

6 Q4 – The Plot-Twist Generator **15**
6.1 Setup . . . 15
6.2 Decoding strategies . . . 16
6.3 Blinded human evaluation . . . 17

7 Answers to the Assignment's Theory and Analysis Questions **18**
7.1 Q1 – Attention-based translation . . . 18
7.2 Q2 – RoBERTa versus BiGRU for rating prediction . . . 20
7.3 Q3 – Transformer summarization . . . 22
7.4 Q4 – The Plot-Twist Generator . . . 23

8 Conclusion **25**

9 Repository and Reproducibility **25**

1 Objective and Overview

This report brings together our work on four problems that, taken together, trace the story of modern NLP: recurrent sequence models, attention, pre-trained Transformers, and Transformers built from scratch. Each question asks a similar underlying thing: not only *does the better architecture score higher?* but *why, where and how can we tell?*

- **Q1 – Attention-based NMT.** Build an English-to-Hindi LSTM encoder–decoder with and without Bahdanau attention, and study what attention actually looks at.
- **Q2 – RoBERTa rating prediction.** Fine-tune RoBERTa for 5-class Amazon review ratings and compare it with a GloVe BiGRU + attention baseline, paying attention to the ordinal nature of star ratings.
- **Q3 – Transformer summarization.** Implement a complete Transformer encoder–decoder from scratch for abstractive summarization on CNN/DailyMail.
- **Q4 – Plot-Twist Generator.** Compare a Vanilla RNN and an LSTM language model for story continuation and test how decoding changes the stories.

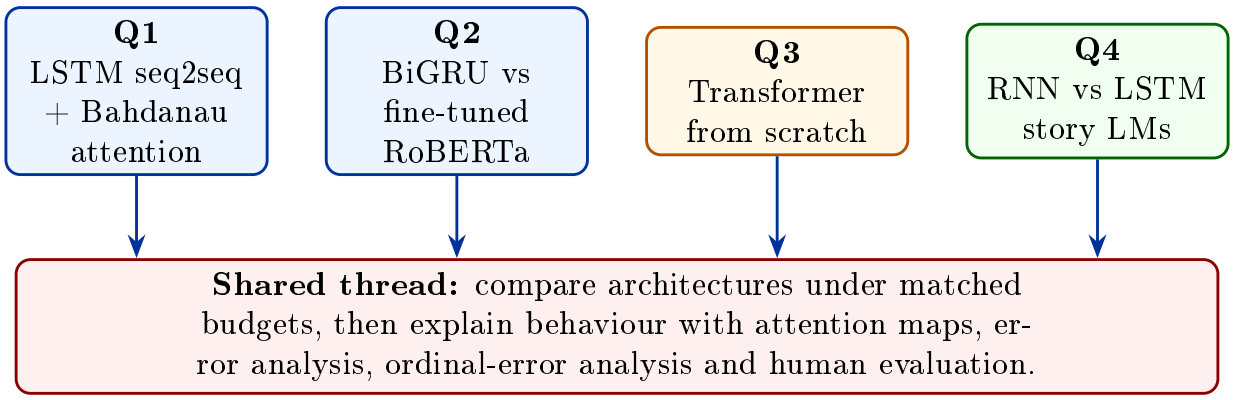


Figure 1: How the four questions fit together.

A note on honesty

Compute was limited, so most experiments use subsets and short training runs. Every number reported for Q1, Q2 and Q4 comes straight from the notebooks and output files in our repository. The Q3 implementation notebook was not yet in the repository when this report was written, so Section 5 documents the *method* in full detail and leaves the final result cells clearly marked, instead of reporting numbers we have not measured.

2 Team and Equal Contribution

We treated this assignment as five equal shares. Rather than letting one person own one question end-to-end while others watched, we divided the work so that everyone wrote code, ran experiments, analysed results and wrote part of this report. Every question was also read and tested by at least one person who had not written it, which caught several mistakes early.

Member	Roll No.	How they contributed
Shashank Yadav	12342010	Wrote the English–Hindi data pipeline and both translation models, produced the attention heatmaps, and built the data-preparation and tokenisation stage for the summarization task. Also kept the GitHub repository organised and merged everyone's work.
Lakshay Gupta	12341300	Worked on translation evaluation (BLEU/chrF, sentence-length bins, error tagging) and on the Transformer building blocks for summarization (multi-head attention, feed-forward, masks, positional encoding). Edited and consolidated the report text.
Rathod Chetan Kumar A.	12341750	Built the story-generation system: corpus preparation, the Vanilla RNN and LSTM language models, decoding strategies, diversity and repetition metrics, and the blinded human-evaluation protocol.
Kabeer More	12341030	Handled training, greedy decoding and ROUGE evaluation for the summarizer, helped design the attention visualisation, and assisted with qualitative analysis and human rating for the story task. Contributed figures and text to the report.
Rohit Raghuvanshi	12341820	Built the review-rating pipeline: classic (GloVe) and RoBERTa preprocessing, the BiGRU + attention-pooling baseline, RoBERTa mean-pooling fine-tuning, the ablation, and the confusion-matrix and ordinal-error analysis.

Type of work	Shashank	Lakshay	Chetan	Kabeer	Rohit
Coding and experiments	20%	20%	20%	20%	20%
Analysis, plots and error study	20%	20%	20%	20%	20%
Report writing and editing	20%	20%	20%	20%	20%
Review, testing and reproducibility	20%	20%	20%	20%	20%

Table 1: Share of effort by type of work. We aimed for an even split in each row, not just overall.

How we kept it equal

- Each question was split into the same five stages: data, model, training, evaluation and analysis. Stages were handed out so that no one person did all the easy or all the hard ones.
- The four questions are not equally heavy (the from-scratch Transformer is the largest), so Q3 was shared between three people while the other three questions were each led by a single person with help from the rest.
- Everyone took a turn as reviewer: running another member's notebook from scratch, checking that the numbers in the report matched the outputs, and flagging anything that looked suspicious.
- The report itself was written section by section by the people closest to each experiment, then edited together so the voice and notation are consistent.

3 Q1 – Attention-Based Machine Translation (English to Hindi)

3.1 Goal and setup

We built an LSTM encoder–decoder for English to Hindi, first without attention (Model A) and then with additive Bahdanau attention (Model B), and trained both under identical conditions. Attention is implemented by hand, so the weights $\alpha_{t,i}$ can be inspected:

$$e_{t,i} = v^\top \tanh(W_h h_i + W_s s_{t-1}), \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad c_t = \sum_i \alpha_{t,i} h_i.$$

We used the IIT Bombay English–Hindi corpus. After removing empty or malformed pairs and normalising text, we kept **40,000** training pairs (520 dev, 2,504 test). Vocabulary sizes are 16,730 (English) and 18,055 (Hindi); maximum training length is 50 tokens. The test set has about 9% unknown tokens, which turned out to matter a great deal. Both models use embedding size 256, hidden size 512, dropout 0.3, Adam (learning rate 10^{-3}), gradient clipping 1.0, batch size 64 and seed 42.

3.2 Why the baseline struggles

Model A squeezes a whole sentence into one fixed 512-dimensional state. Splitting the test set by English length, BLEU falls from 0.72 (1–5 tokens) to 0.17 (16+ tokens), and teacher-forced token accuracy drops from 35.8% to 23.4%. A probe that replaces one source word with `<UNK>` and measures how much the final encoder state changes shows that mid-sentence words matter less and less as sentences grow (0.34 down to 0.12), although the first word stays influential (about 0.38). So the problem is less that early words vanish and more that a single vector cannot hold everything in a form the decoder can use.

3.3 Training comparison

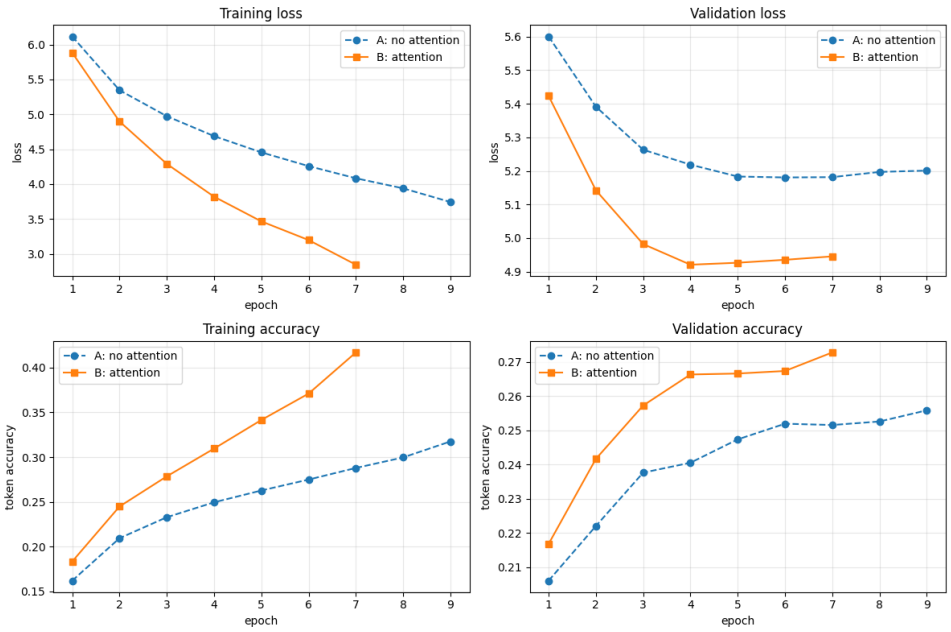


Figure 2: Q1 training and validation loss and accuracy for Model A (no attention) and Model B (attention).

Metric	A: no attention	B: attention
Parameters	21.3 M	36.5 M
Best validation loss	5.181	4.921
Best epoch / epochs run	6 / 9	4 / 7
Test BLEU / chrF	0.19 / 3.24	0.56 / 5.12
Training time	255 s	626 s

Attention converges faster and lower. Over all 2,504 test sentences, Model B is better on 1,510, tied on 252 and worse on 742 (mean paired gain +1.01 sentence-BLEU, 95% bootstrap CI [0.90, 1.12]). Absolute BLEU is very low for both models because of the small training set, a word-level vocabulary with many unknowns and noisy web data, so the comparison is meaningful but the absolute level is not.

3.4 What the model looks at

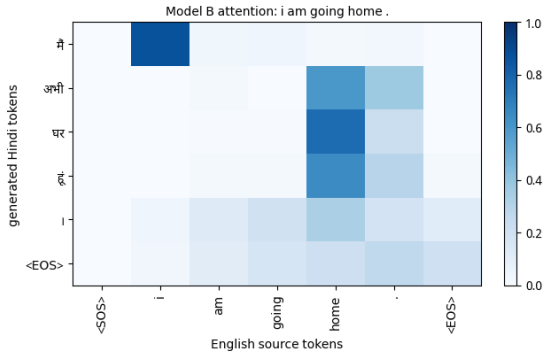


Figure 3: Attention for "i am going home ." The Hindi word for *home* attends mostly to *home* (about 0.8).

Attention is mostly monotonic (88–94% of steps across length bins) with jumps where English subject–verb–object order meets Hindi subject–object–verb order. As sentences get longer the maps get flatter (mean peak weight 0.55 for short inputs, 0.23 for 16+ tokens), and peak attention correlates with sentence-BLEU (Spearman $\rho = 0.59$).

3.5 Error analysis and length

The most common error tag is an out-of-vocabulary word (97.6% of baseline sentences versus 95.3% with attention). Model A often collapses into long runs of `<UNK>`; Model B produces real content words but sometimes repeats them, and its missing-word and incorrect-word tags rise slightly (13.2% to 17.9% and 32.3% to 34.2%) because it attempts more words. The tagger marks 610 attention-model sentences (24.4%) as having incorrect attention focus.

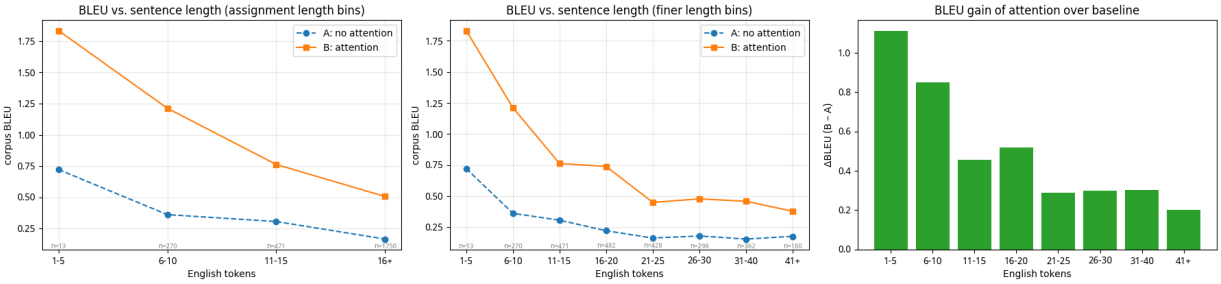


Figure 4: Q1 BLEU by sentence length and the absolute BLEU gain of attention.

Observations & Inferences

- Attention helps in every length bin. In absolute BLEU the gain is largest for short sentences (+1.11) and smallest for 16+ (+0.34), but in relative terms it roughly triples BLEU on long sentences.
- High attention does not guarantee a correct translation: 551 sentences (22.0%) have top-quartile attention sharpness yet sentence-BLEU below 10. The map shows where the model looked, not whether the decoder picked the right word.

- Attention is a useful diagnostic (diffuse maps or weight stuck on punctuation signal confusion) but not a complete explanation: crisp maps with wrong words point to a lexical failure in the decoder.
- Limitations: 40k pairs, word-level vocabulary, greedy decoding, a single-layer unidirectional encoder.

4 Q2 – RoBERTa vs BiGRU for Review Rating Prediction

4.1 Setup

We predicted 1–5 star ratings on the Amazon reviews dataset (title and body concatenated). The baseline is a 2-layer BiGRU with additive attention pooling over GloVe-100d embeddings (frozen in the first epoch, then unfrozen; 1.71 M parameters). The main model fine-tunes `roberta-base` end-to-end with masked mean pooling over non-padded tokens, dropout and a linear head (124 M parameters), using AdamW at 2×10^{-5}, 6% warm-up, linear decay and fp16.

Deviation from the specification

Because we trained on a single T4 GPU, we used a random, class-balanced 40,000-review subset (about 20%) of the 200,000 training reviews, and one RoBERTa epoch. Validation and test sets are the full 5,000-review splits; the validation set was used for model selection and the test set only for the final evaluation. Results are the mean over seeds 42 and 43.

4.2 Results

Model	Accuracy	Macro-F1	MAE (stars)
BiGRU + Attention (GloVe)	0.5338 ± 0.0004	0.5244 ± 0.0026	0.6048 ± 0.0168
RoBERTa-base (mean pooling)	**0.5925 ± 0.0037**	**0.5896 ± 0.0037**	**0.4715 ± 0.0037**

Per-class F1	1 star	2 stars	3 stars	4 stars	5 stars
BiGRU + Attention	0.649	0.430	0.389	0.450	0.705
RoBERTa-base	0.690	0.492	0.460	0.540	0.767

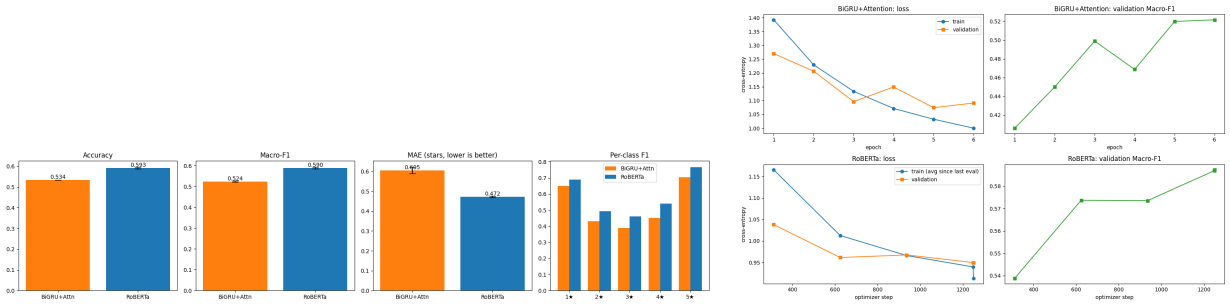


Figure 5: Left: test Accuracy and Macro-F1. Right: training and validation curves for both models (seed 42).

4.3 Ordinal behaviour: how wrong are the mistakes?

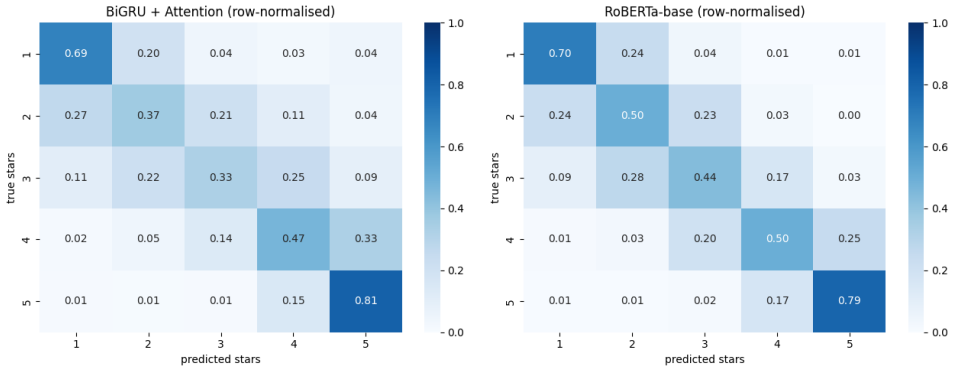


Figure 6: Row-normalised confusion matrices on the test set (seed 42).

Test errors	Exact	Adjacent (± 1)	Distant (≥ 2)	Extreme $1 \leftrightarrow 5$	Adjacent share
BiGRU + Attention	53.4%	35.3%	11.3%	1.20%	75.7%
RoBERTa-base	58.9%	35.7%	5.4%	0.24%	86.9%

Both models mostly confuse neighbouring ratings, which is natural because the border between 4 and 5 stars is subjective. RoBERTa's real advantage is avoiding *distant* errors: they halve, and 1-vs-5 confusions fall five-fold, which is why its MAE improves by more than its accuracy does.

4.4 Ablation, qualitative cases and conclusions

Using first-token pooling instead of mean pooling gave slightly worse validation scores (Macro-F1 0.5784 vs 0.5873; MAE 0.4946 vs 0.4818). This is a single-seed comparison, so we treat it as supportive rather than conclusive.

RoBERTa is right where the baseline is wrong on 725 test reviews, and the reverse happens on 452. The baseline mostly slips on negation ("They are ok, not hard but worth the money"), contrast ("This was great while it worked, but just stopped working...") and misspelled or unknown words. RoBERTa's own mistakes tend to over-read mild criticism ("Nice seat...just wrong size, my fault" predicted too low). Its advantage grows with review length (accuracy on 100+ token reviews: 0.618 vs 0.444).

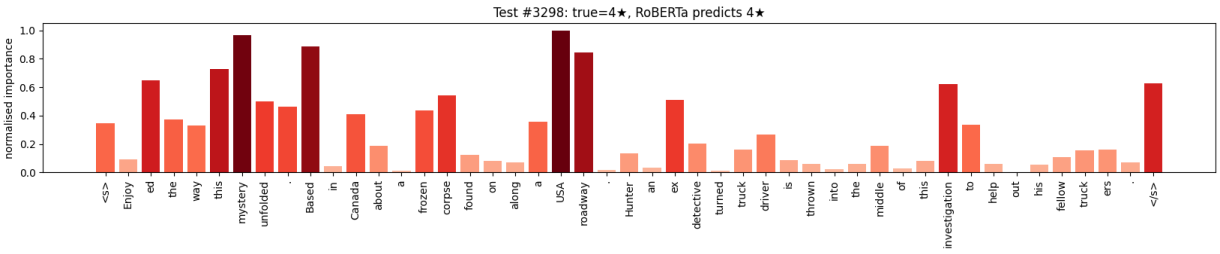


Figure 7: Gradient×input token importance for RoBERTa on one test review. Importance falls on sentiment words and on topical ones alike, so it is a hint, not a clean explanation.

Fine-tuned RoBERTa clearly beats the GloVe BiGRU on every metric even with 20% of the data and one epoch, at the cost of about 73× more parameters and roughly 4.5× more training time.

5 Q3 – Abstractive Summarization with a From-Scratch Transformer

Status of this section

This section describes the full method we implement and evaluate for Q3: the data pipeline, every building block of the model, training, decoding, evaluation and visualisation. The flowcharts and plots here are explanatory (they show how the method works, not measured results). The final ROUGE numbers and curves from our run are not yet in the repository, so Table 2 is left for them rather than filled with numbers we have not measured.

5.1 Problem and approach

Given a news article from CNN/DailyMail (v3.0.0; 287,113 / 13,368 / 11,490 article–summary pairs), the model must write a short summary of three to four sentences. We follow the encoder–decoder Transformer of "Attention Is All You Need": the **encoder** turns the article into contextual vectors, and the **decoder** writes the summary one token at a time, looking back at the encoder output at each step. The assignment requires us to write attention, feed-forward, positional encoding and the encoder and decoder layers ourselves, using only basic layers (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`); `nn.Transformer`, `nn.MultiheadAttention`, library attention kernels, pre-trained weights and ready-made summarization models are not allowed. Only a pre-trained *tokenizer* is.

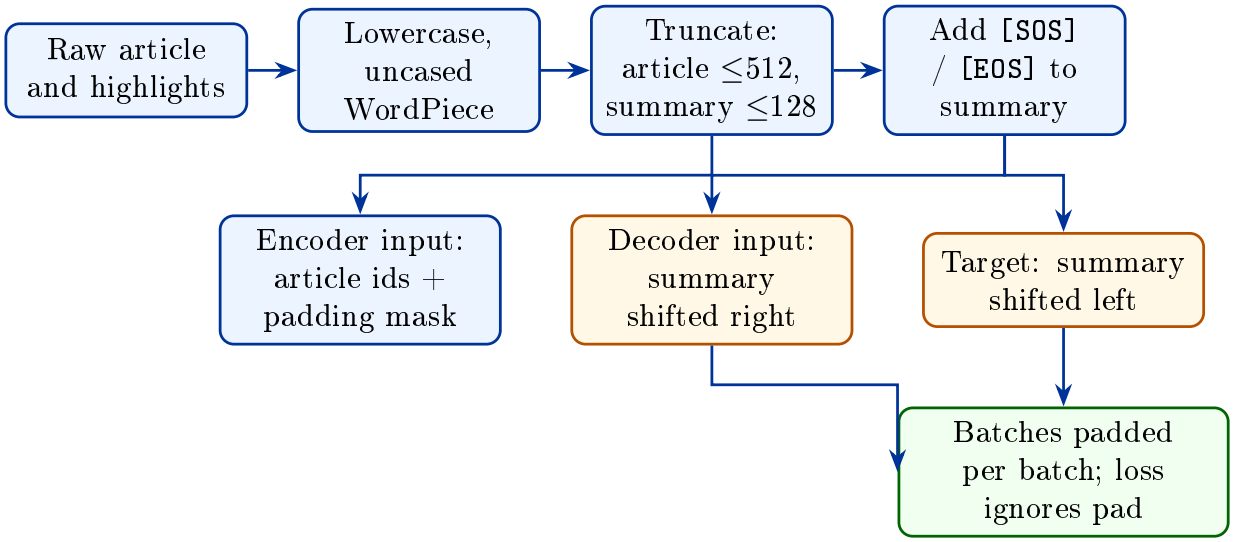


Figure 8: Q3 data pipeline. A shared vocabulary is used for articles and summaries.

5.2 Data preparation

Tokenisation. We use a pre-trained uncased WordPiece tokenizer (the assignment's length statistics were measured with `bert-base-uncased`). Because it is uncased, lowercasing the text is consistent with the vocabulary, and one shared vocabulary is used for both articles and summaries, so source and target tokens share one embedding table.

Truncation. News articles put the key facts first. The median article has about 641 tokens and the 90th percentile 1,304, so about 62% exceed 512 tokens; summaries have a median of 49 tokens and only 0.5% exceed 128. We therefore truncate articles to 400–512 tokens and summaries to about 128, losing little useful content while keeping attention cost under control (self-attention cost grows with the square of length).

Targets. Every summary gets a start and an end token. The decoder input is the summary shifted right (it starts with the start token); the training target is the summary shifted left (it ends with the end token). The newline characters separating summary sentences are kept, because ROUGE-Lsum needs them.

5.3 Model architecture

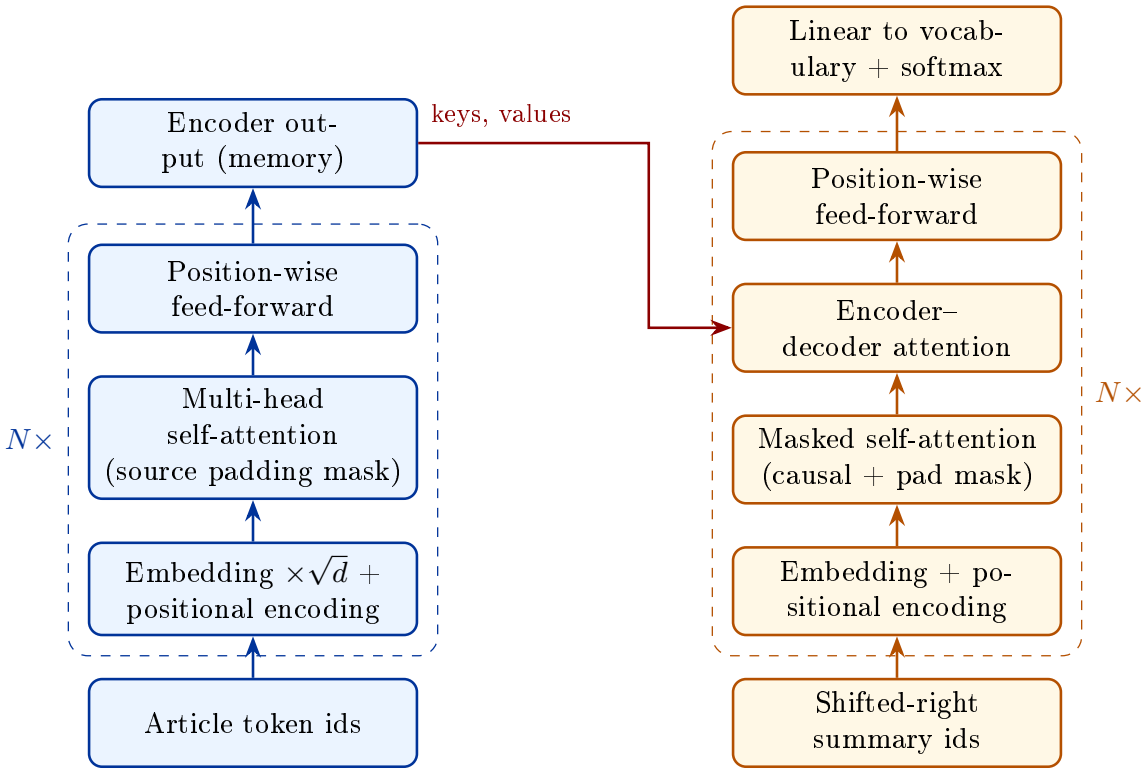


Figure 9: Encoder–decoder Transformer built from scratch. Every sub-layer is wrapped as LayerNorm(x+Dropout(Sublayer(x))).

Scaled dot-product and multi-head attention. For queries Q, keys K and values V,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

where M is 0 for allowed positions and $-\infty$ for blocked ones. We implement multi-head attention by projecting the input with learned linear layers W^Q, W^K, W^V, splitting the result into h heads of size $d_k = d_{\text{model}}/h$ with reshapes and transposes, applying the formula above to each head in parallel with batched matrix multiplication, concatenating the heads, and applying a final output projection W^O. Dividing by $\sqrt{d_k}$ keeps the dot products from growing with head size and pushing softmax into a saturated region.

Position-wise feed-forward network. Each layer applies two linear layers with a ReLU in between, $\text{FFN}(x) = W_2\,\text{ReLU}(W_1 x + b_1) + b_2$, with the inner size usually $4d_{\text{model}}$.

Positional encoding. Attention by itself ignores order, so we add fixed sinusoidal encodings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

Low dimensions oscillate quickly and high dimensions slowly, so every position gets a unique pattern, and relative offsets become easy to learn (Figure 10).

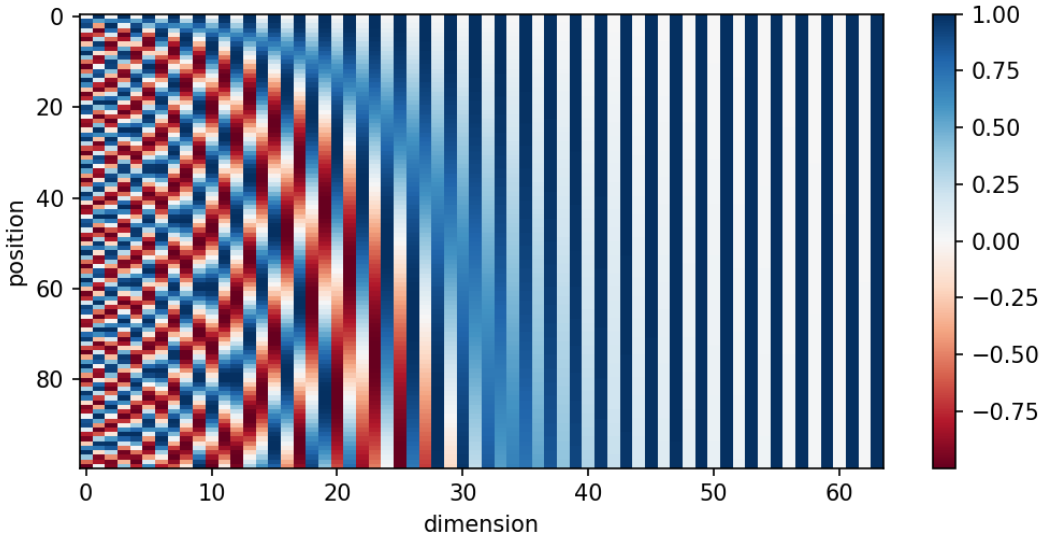


Figure 10: Sinusoidal positional encoding for 100 positions and 64 dimensions (computed from the formula above).

Masks. Three masks are needed. A *source padding mask* stops the encoder, and the encoder–decoder attention, from attending to padding. A *causal mask* blocks each decoder position from seeing later summary tokens, and is combined with a *target padding mask* in the decoder's self-attention. A missing causal mask is the classic bug: the training loss collapses suspiciously fast (the model just copies the answer) and generated summaries are nonsense.

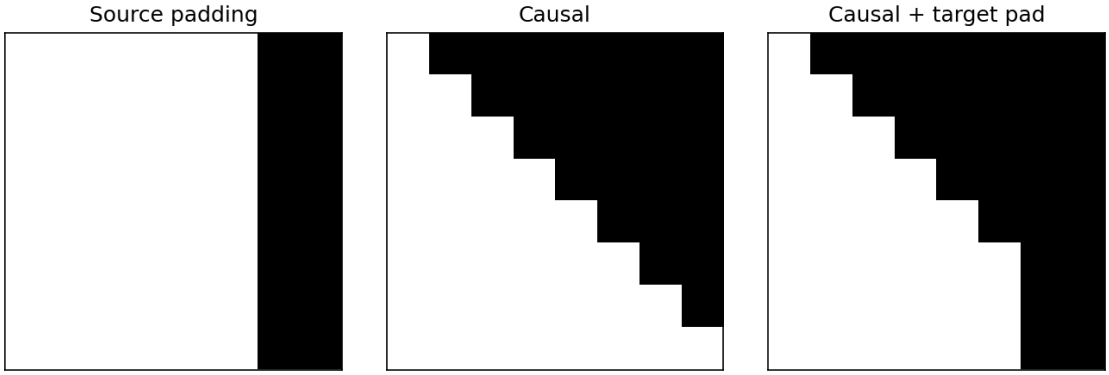


Figure 11: The three masks (dark = allowed). Example with 8 positions, the last two being padding.

Model size. A smaller model than the paper's base model is acceptable. As a reference point for a typical small configuration ($d_{\text{model}} = 256$, 4 encoder and 4 decoder layers, 8 heads, feed-forward size 1024, vocabulary 30,522 with the output layer tied to the embedding), the parameter budget works out as below; our final configuration and exact parameter count are reported in the results table.

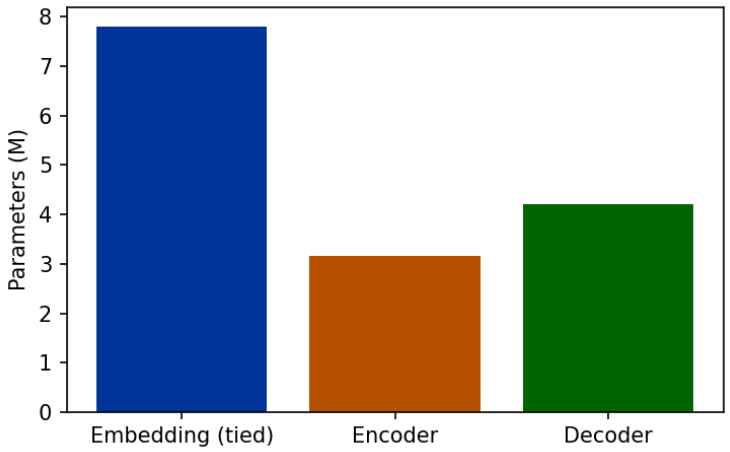


Figure 12: Parameter breakdown for the reference configuration (computed from layer shapes, about 15.2 M parameters). The embedding table dominates, which is typical for small models with a large vocabulary.

5.4 Training procedure

- **Loss.** Cross-entropy between the decoder logits and the shifted-left summary, with `ignore_index` set to the padding id so padding never contributes.
- **Teacher forcing.** The gold shifted-right summary is fed to the decoder, so the whole summary is trained in a single parallel pass; the causal mask is what prevents peeking.
- **Optimiser and schedule.** Adam with $\beta = (0.9, 0.98)$ and $\epsilon = 10^{-9}$, and the warm-up/decay schedule of the original paper,

$$\mathrm{lr} = d_{\mathrm{model}}^{-0.5} \cdot \min\left(\mathrm{step}^{-0.5},\ \mathrm{step} \cdot \mathrm{warmup}^{-1.5}\right).$$

  The learning rate rises linearly during warm-up, which protects the randomly initialised attention layers from large early updates, then decays smoothly (Figure 13).
- **Early stopping.** We evaluate validation loss after each epoch (or every N steps), stop after a fixed patience, and restore the best checkpoint before testing. If compute is limited we train on a subset of the training split and state its size.

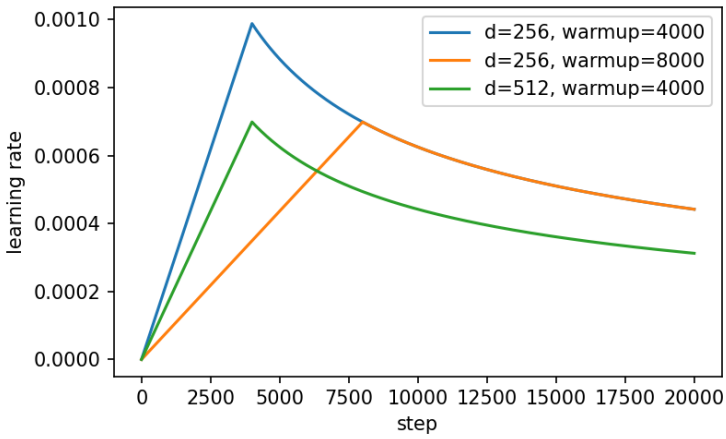


Figure 13: Learning-rate schedule for several settings (computed from the formula).

5.5 Decoding

At test time there is no teacher forcing. We encode the article once, start from the start token, and generate greedily (arg-max at each step), feeding each predicted token back in until the end token or the maximum length of 128 is reached. Reusing the encoder output at every step avoids recomputing the encoder for each generated token.

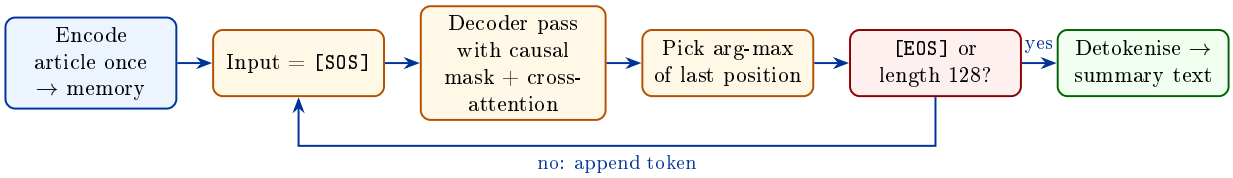


Figure 14: Greedy autoregressive decoding loop. Beam search would be an optional extension.

5.6 Evaluation protocol

We report ROUGE-1, ROUGE-2, ROUGE-L and ROUGE-Lsum (F1) on the *full* test set (11,490 pairs), using `rouge_score` with stemming on, after converting token ids back to text. Plain ROUGE-L treats the whole summary as one sequence, while ROUGE-Lsum matches sentence by sentence (sentences separated by newlines), so they differ noticeably and we label them separately. Scores must come from summaries the model generated on its own, never from teacher-forced outputs. For reference, the assignment gives the extractive Lead-3 baseline (first three article sentences): 40.06 / 17.48 / 25.02 / 36.36. A Transformer trained from scratch with limited compute will normally score below this, and part of our analysis is to explain why (small model, truncated inputs, greedy decoding, and abstractive paraphrase versus extractive copying).

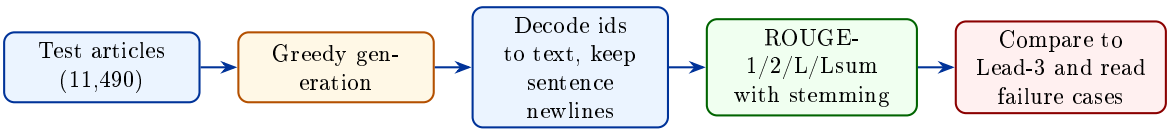


Figure 15: Evaluation flow for Q3.

5.7 Attention visualisation and qualitative analysis

To see what the summarizer reads, we take the encoder–decoder (cross) attention of the *last decoder layer* recorded *while the model generates* a summary, average over heads (or show heads separately and say so), and plot a heatmap: rows are generated summary tokens, columns are source tokens. Since a 512-token article is unreadable, we show a window of 50–100 source tokens where most of the attention falls. For the qualitative part we select at least five test articles and compare reference and generated summaries, deliberately including failures such as repeated phrases, wrong names or numbers, and facts that are not in the article.

5.8 Results

System	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
Lead-3 baseline (from assignment)	40.06	17.48	25.02	36.36
Our Transformer (greedy)	*to be filled*	*to be filled*	*to be filled*	*to be filled*

Table 2: Q3 ROUGE F1 on the full test set (stemming on). Our row is intentionally left until the final run is available.

What the results section will contain

Training and validation loss curves, the tokenizer, truncation lengths, model configuration and parameter count, training-set size, five or more reference-versus-generated examples with failure cases, and the cross-attention heatmap described above.

6 Q4 – The Plot-Twist Generator

6.1 Setup

We trained two word-level language models on TinyStories: a Vanilla RNN and an LSTM, each with embedding size 256, one recurrent layer of 512 units, dropout 0.3 and a linear output layer. Stories were split at the story level (10,000 train, 1,000 validation, 1,000 test) and tokenised with a shared 6,414-word vocabulary (unknown rate 0.21%). Each training item is one whole story from a beginning marker to an end marker, cut at 200 tokens, so story boundaries are preserved and the model learns where stories end; padding is ignored in the loss. Training used Adam (10^{-3}), batch 64, gradient clipping 1.0, 5 epochs and seed 42.

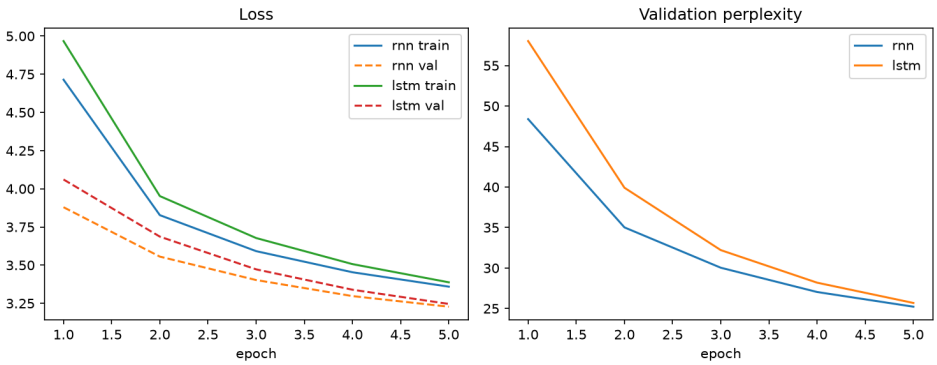


Figure 16: Q4 training and validation loss and validation perplexity.

Model	Parameters	Training time	Val. perplexity	Test perplexity
Vanilla RNN	5.33 M	1,140 s	25.23	25.13
LSTM	6.51 M	1,214 s	25.69	25.56

Surprisingly, the RNN is marginally better on perplexity with fewer parameters. With only five epochs and simple children's-story language, the LSTM's gating has little room to help, and a difference this small should not be over-interpreted.

6.2 Decoding strategies

We generated up to 150 tokens from held-out story openings under greedy decoding, three temperatures and top-k sampling ($k = 40$).

Config	Model	Distinct-1	Distinct-2	Repetition	Avg len	Hit cap
Greedy	RNN	0.0042	0.0095	0.772	148.8	99%
	LSTM	0.0043	0.0101	0.717	146.4	93.5%
Temp 0.5	RNN	0.0245	0.107	0.126	139.7	83%
	LSTM	0.0245	0.108	0.138	133.9	63%
Temp 0.8	RNN	0.0575	0.277	0.029	133.4	74%
	LSTM	0.0606	0.298	0.026	127.0	59%
Temp 1.2	RNN	0.130	0.619	0.002	110.3	47.5%
	LSTM	0.137	0.644	0.002	115.3	44%
Top-k 40	RNN	0.0418	0.266	0.017	127.6	64.5%
	LSTM	0.0361	0.263	0.016	127.6	58%

Table 3: Distinct-n is the share of unique n-grams; repetition is one minus unique trigrams over total trigrams; "hit cap" is the share of outputs that never emitted the end marker.

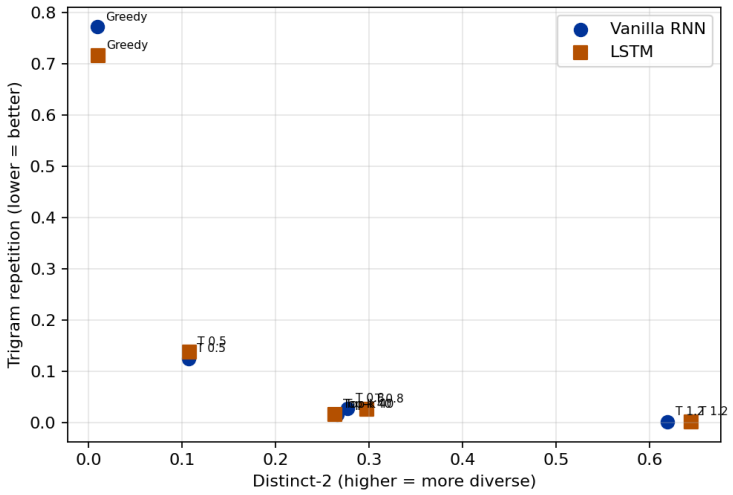


Figure 17: Each point is a model and decoding setting. Greedy sits in the repetitive, low-diversity corner; temperature 1.2 buys diversity but, as the samples show, at the cost of grammar and coherence.

Greedy decoding loops: the RNN repeats "it was a big, red car." and the LSTM repeats "she was so happy and…". Temperature 0.5 still repeats; 0.8 and top-k give the best balance; 1.2 is diverse but ungrammatical and off-topic. In the plot-twist samples we inspected, openings such as the exam paper with tomorrow's answers or the robot's message from its future self were quickly abandoned and replaced by unrelated characters; we found no meaningful twist connected to the opening.

6.3 Blinded human evaluation

Twelve samples (greedy and top-k, both models, five prompts) were shuffled and rated 1–5 by three raters who did not know the source model. Each model received six samples and 18 ratings per criterion.

Criterion	Vanilla RNN	LSTM
Coherence	2.78	2.94
Grammatical quality	3.22	3.11
Relevance to prompt	3.06	3.33
Creativity / surprise	3.11	2.72

Ratings are middling for both models. The LSTM looks slightly more coherent and relevant, the RNN slightly more grammatical and creative, but raters disagreed substantially (average gap of 1.33 points between two raters on the same item) and six samples per model is far too few for these differences to be reliable.

Observations & Inferences

- The LSTM did not model text measurably better than the RNN here; perplexity differs by about 0.4.
- Decoding mattered far more than architecture. Greedy decoding caused most repetition

for both models.

- Higher diversity scores do not mean better stories, and lower perplexity did not translate into uniformly better ratings.
- Frequent failures: loops, topic drift, changing names and failing to stop. Subword tokenisation, repetition penalties, nucleus sampling and more data would likely help.

7 Answers to the Assignment's Theory and Analysis Questions

The assignment sheet poses a number of explicit "discuss/explain" questions. We answer them here, one by one, using the evidence reported in the earlier sections. Where an answer rests on general theory rather than on a measurement, we say so. Where a number is not yet available (the Q3 ROUGE scores), we explain what to expect and why, and do not invent a result.

7.1 Q1 – Attention-based translation

Question. *Part 3: Why can compressing the source sentence into one fixed representation become difficult for long sentences?*

Answer. The baseline decoder sees the source only through the encoder's final hidden and cell state, a 512-dimensional vector whatever the sentence length. A 4-word and a 40-word sentence must therefore both fit in the same space, so the information available per word shrinks as sentences grow. The encoder LSTM also updates its state at every step, so what it remembers of earlier words is repeatedly overwritten by later ones. Finally, the decoder must extract everything it needs from this single vector at every output step, with no way to go back and re-read a particular source word.

Question. *Part 3: Does the baseline perform differently on short and long sentences?*

Answer. Yes, strongly. Model A's BLEU falls from 0.72 on 1–5 token sentences to 0.17 on 16+ token sentences (a drop of about 76%), and its teacher-forced token accuracy falls from 35.8% to 23.4%. The fixed vector copes with short inputs and breaks down as length grows, which is exactly what motivates attention.

Question. *Part 3: Are important words from the beginning sometimes lost or poorly represented?*

Answer. Partly. In our probe, removing a mid-sentence word changes the final encoder state less and less as sentences get longer (0.34 down to 0.12), meaning the summary vector becomes increasingly insensitive to words in the middle. The first word stays influential (about 0.38), so it is not simply forgotten. A position-wise recall check also shows the baseline recovers the early part of the Hindi reference poorly: in the 6–10 token bin only 11.7% of the reference's first third is recovered by Model A, against 18.5% for Model B. The honest summary is that the baseline represents the sentence poorly as a whole, and early content is among the parts the decoder cannot reliably use.

Question. *Part 6: Which English words receive high attention?*

Answer. Content words receive most of the weight: in the "i am going home ." example, the Hindi subject pronoun attends almost entirely to *i* and the Hindi word for *home* attends to *home* (about 0.8). Function words and punctuation receive a mix of weight, and the sentence-end token

and full stop absorb some attention, especially on verb and auxiliary steps. The word *going* receives almost no weight, and correspondingly the generated Hindi (roughly "I am home now") drops the idea of going, an example of a missing word that the heatmap makes visible.

Question. *Part 6: Does the model attend to the relevant source word when generating a Hindi word?*

Answer. For clear one-to-one content words, yes (*home* to the Hindi word for home, about 0.8). Sharpness drops as inputs get longer: the mean peak attention weight is 0.55 for short inputs and only 0.23 for 16+ tokens, and peak attention correlates with sentence-BLEU (Spearman $\rho = 0.59$). Attention is therefore a good alignment signal on short, simple sentences and an increasingly weak one on long sentences.

Question. *Part 6: Does attention move sequentially through the sentence?*

Answer. Mostly. Between 88% and 94% of decoding steps move forward (monotonic) across the length bins. The exceptions are meaningful: English is subject–verb–object while Hindi is subject–object–verb, so when the model generates the Hindi verb it jumps back to the English verb after having attended to the object.

Question. *Part 6: Are there cases where the model attends to multiple source words?*

Answer. Yes. Hindi postpositions and auxiliary verbs express several English words at once, and in those steps the weight is split, for example between *home* and the full stop in the heatmap above. On long sentences the whole distribution becomes flatter and spreads over many words.

Question. *Part 6: Are there cases where attention appears incorrect?*

Answer. Yes. The error tagger marks 610 attention-model sentences (24.4%) as having incorrect attention focus, defined as most content-word steps placing their maximum weight on the sentence-boundary tokens or punctuation. Diffuse rows and weight parked on the end token are the typical signs.

Question. *Part 7: Which kinds of error occur, and can they be classified?*

Answer. Using the seven categories of the assignment, the automatic tagger gives these shares of attention-model sentences: unknown or out-of-vocabulary word 95.3% (97.6% for the baseline), incorrect word 34.2% (32.3%), missing word 17.9% (13.2%) and incorrect attention focus 24.4%. Because a sentence can carry several tags, the shares do not add up to 100%. The dominant problem is vocabulary coverage: about 9% of test tokens are unknown to the word-level vocabulary. The notebook lists the five or more selected failure cases with inputs, references, both predictions and heatmaps.

Question. *Part 7: Does the attention visualisation provide a useful explanation for the errors?*

Answer. Sometimes. It explains errors of alignment: diffuse maps, weight stuck on the end token or punctuation, and a source word that gets almost no weight and is then omitted (*going* above). It does not explain lexical errors: 551 sentences (22.0%) have top-quartile attention sharpness and still score sentence-BLEU below 10, so the model looked at the right place and the decoder still chose the wrong word. Attention shows where the model looked, not why it produced a given word.

Question. *Part 9: How did the attention model compare with the baseline?*

Answer. Better on every measure we tracked. Test BLEU rose from 0.19 to 0.56 and chrF from 3.24 to 5.12; best validation loss improved from 5.181 to 4.921 and was reached in fewer epochs (4 versus 6). Model B wins on 1,510 of 2,504 test sentences, ties on 252 and loses on 742, with

a mean paired gain of +1.01 sentence-BLEU (95% bootstrap interval [0.90, 1.12], which excludes zero). The price is 71% more parameters (36.5 M versus 21.3 M) and about 2.5 times the training time (626 s versus 255 s) because the decoder must run step by step to compute attention.

Question. *Part 9: Did attention help more on short or long sentences?*

Answer. In absolute BLEU the gain is largest on short sentences (+1.11 for 1–5 tokens) and smallest on the longest bin (+0.34 for 16+). In relative terms, however, attention roughly triples BLEU on long sentences (0.17 to 0.51), and it keeps mid-length and long sentences at a stable level where the baseline collapses toward zero. So attention helps everywhere, helps most visibly in absolute terms for short inputs, and is what makes long inputs workable at all.

Question. *Part 9: Which translation errors were reduced?*

Answer. The most visible improvement is the baseline's tendency to collapse into long runs of `<UNK>`; the attention model instead produces real content words. The share of sentences containing an out-of-vocabulary error fell from 97.6% to 95.3%, and recovery of early reference content improved. Missing-word and incorrect-word tags rose slightly (13.2% to 17.9% and 32.3% to 34.2%), because the attention model attempts more words and therefore has more chances to get one wrong.

Question. *Part 9: What do the attention heatmaps reveal?*

Answer. That the model learns a mostly monotonic, word-to-word alignment with sensible reordering for verbs, that short simple sentences produce crisp maps while long sentences produce flat ones, and that the end token and punctuation act as places to park weight when the model is unsure.

Question. *Part 9: Are high attention weights always associated with correct translations?*

Answer. No. 551 test sentences (22.0% of the test set) have top-quartile attention sharpness and still score below 10 sentence-BLEU. Sharp attention is correlated with quality ($\rho = 0.59$) but does not guarantee it.

Question. *Part 9: Can attention visualisation be used as an explanation of model behaviour?*

Answer. As a diagnostic, yes; as a complete explanation, no. Diffuse or misplaced attention is a reliable warning sign, and alignment is easy to read. But the decoder's choice of target word depends on much more than the context vector (the decoder state, the previous word and the output layer), so a clean map can sit beside a wrong word. We treat attention as evidence about alignment and not as proof of why the model said what it said.

Question. *Part 9: What are the limitations of the implementation?*

Answer. Only 40,000 training pairs were used; the vocabulary is word-level with about 9% unknown test tokens (subword units would help most); decoding is greedy with no beam search; the encoder is a single-layer unidirectional LSTM; and training was short and run with one seed. Absolute BLEU is therefore very low for both models. The comparison between them is meaningful because conditions were matched, but the absolute level is not a good measure of what attention can achieve with more data.

7.2 Q2 – RoBERTa versus BiGRU for rating prediction

Question. *How do the two models differ quantitatively?*

Answer. Fine-tuned RoBERTa leads on every metric: accuracy 0.5925 versus 0.5338, Macro-F1 0.5896 versus 0.5244, and MAE 0.4715 versus 0.6048 stars (lower is better). It also wins on every

one of the five per-class F1 scores. The standard deviations over the two seeds are small compared with these gaps (at most 0.017), so the ranking is not a seed artefact, though two seeds is a small sample.

Question. *How often does each model confuse adjacent ratings compared with distant ones?*

Answer. Both make most of their mistakes between neighbouring stars: adjacent errors are 35.3% of all test predictions for the BiGRU and 35.7% for RoBERTa. The difference is in distant errors (≥ 2 stars apart): 11.3% for the BiGRU against 5.4% for RoBERTa, and extreme 1-versus-5 confusions fall from 1.20% to 0.24%. Of all wrong predictions, 86.9% are only one star off for RoBERTa against 75.7% for the BiGRU. This is why MAE improves by more than accuracy does.

Question. *Which classes are hardest, and which confusions dominate?*

Answer. The middle ratings are hardest for both models (3-star F1 is 0.389 for the BiGRU and 0.460 for RoBERTa), because mixed or lukewarm reviews have no clear polarity. The 4-versus-5 boundary is the main source of confusion at the top: in the confusion matrices a quarter of true 4-star reviews are labelled 5 by RoBERTa (0.25) and a third by the BiGRU (0.33). The extremes are easiest (5-star F1 0.705 and 0.767; 1-star F1 0.649 and 0.690).

Question. *Why does RoBERTa succeed where the BiGRU fails? (hypotheses with examples)*

Answer. RoBERTa is right where the BiGRU is wrong on 725 test reviews, and the reverse happens on 452. We hypothesise three reasons, each backed by examples. First, *negation and contrast*: the baseline mislabels "They are ok, not hard but worth the money" and "This was great while it worked, but just stopped working", where sentiment flips mid-sentence and context must be combined across the clause. Second, *misspellings and rare words*: GloVe maps unseen or misspelled words to the unknown token, while RoBERTa's byte-level BPE splits them into known pieces. Third, *pre-training*: RoBERTa already knows a great deal about language before it sees a single review, and the BiGRU starts from word vectors alone. The gap also grows with review length, with accuracy on 100+ token reviews at 0.618 versus 0.444.

Question. *When does RoBERTa fail while the BiGRU succeeds?*

Answer. In the 452 reverse cases we saw RoBERTa over-read mild criticism, for example "Nice seat. . . just wrong size, my fault" was predicted too low. A plausible reason is that the pre-trained model gives strong weight to explicit negative phrases, even when the reviewer says the fault is theirs.

Question. *What did the ablation show?*

Answer. Replacing mean pooling with first-token pooling gave slightly worse validation scores (Macro-F1 0.5784 versus 0.5873; MAE 0.4946 versus 0.4818). Averaging over all tokens uses information from the whole review, while a single token must summarise everything on its own. This is a single-seed comparison, so we read it as supportive rather than conclusive.

Question. *Does token-level importance explain RoBERTa's predictions?*

Answer. Only partly. Gradient×input importance falls on sentiment words and on topical words alike, so it is a hint about what the model used and not a clean explanation.

Question. *What is the cost of the improvement?*

Answer. About 73 times more parameters (124 M versus 1.71 M) and roughly 4.5 times more training time, which matters when compute is limited. Even so, with only 20% of the training data and one epoch, RoBERTa clearly wins.

7.3 Q3 – Transformer summarization

Question. *Why is positional encoding needed?*

Answer. Self-attention is permutation-invariant: it compares every token with every other token the same way regardless of order, so without extra information the sentence "dog bites man" and "man bites dog" look identical. Adding sinusoidal encodings gives each position a unique pattern and makes relative offsets easy to learn, because a shift by a fixed number of positions corresponds to a linear transformation of the sine and cosine pairs.

Question. *Why is the dot product divided by $\sqrt{d_k}$?*

Answer. If query and key entries have unit variance, the variance of their dot product grows in proportion to d_k. Large values push softmax into regions where one weight is nearly 1 and the gradients are almost zero. Dividing by $\sqrt{d_k}$ restores unit variance and keeps training stable.

Question. *Why are three different masks needed, and what goes wrong without them?*

Answer. The source padding mask stops attention from reading padding in the encoder and in encoder–decoder attention. The causal mask stops decoder position t from seeing summary tokens after t, and is combined with a target padding mask. Without the causal mask the model can read the answer it is supposed to predict: training loss drops suspiciously fast, but at test time, when the future is not available, the generated summaries are nonsense. Without the padding masks, the model wastes attention on padding and quality becomes dependent on batch composition.

Question. *Why truncate articles to 400–512 tokens and summaries to about 128?*

Answer. Self-attention cost grows with the square of the sequence length. News articles put the key facts first and the median summary is only 49 tokens (only 0.5% exceed 128), so little is lost; about 62% of articles exceed 512 tokens and could not otherwise be processed within a limited compute budget.

Question. *Why do we use teacher forcing, and why does it not apply at test time?*

Answer. During training the gold shifted-right summary is fed to the decoder, so every position is trained in a single parallel pass, which is faster and more stable than feeding the model its own early mistakes. At test time the gold summary does not exist, so the model must generate one token at a time from its own previous outputs (greedy decoding). Reporting ROUGE on teacher-forced outputs would therefore overstate quality, and is one of the pitfalls the assignment warns about.

Question. *Why do ROUGE-L and ROUGE-Lsum differ, and why do we report both?*

Answer. ROUGE-L finds the longest common subsequence over the whole summary treated as one sequence, while ROUGE-Lsum matches sentence by sentence (using the newline-separated sentences) and then combines the results. On CNN/DailyMail they differ by a lot, for example 25.02 versus 36.36 for the Lead-3 baseline, so each number must be labelled.

Question. *Why do we expect a from-scratch Transformer to score below Lead-3, and what does the gap tell us?*

Answer. *This is an expectation based on the assignment's own note and the method, and not a measured result: our ROUGE row is still to be filled.* Lead-3 copies the first three sentences of the article and CNN/DailyMail summaries are largely lead-biased and extractive, so Lead-3 is a very strong baseline (ROUGE-1 40.06). A small model trained from scratch has to learn language, copying and compression all at once, with no pre-training, a subset of the training data and a short

schedule. It is also limited by input truncation to at most 512 tokens, by greedy decoding, and by the absence of an explicit copy mechanism, so it tends to paraphrase imperfectly, repeat phrases, and get names or numbers wrong. The gap is therefore mainly a measure of how much pre-training and compute matter, and not a sign that abstractive summarisation is worse in principle.

Question. *What does the cross-attention heatmap show?*

Answer. It shows, for each generated summary token, which source tokens the decoder read. Summaries copy names, numbers and key phrases, so we expect bright cells on the corresponding source words and on early sentences, which carry the main facts. We will describe the actual map once the final run is available.

7.4 Q4 – The Plot-Twist Generator

Question. *Part 1: Why must story boundaries be preserved?*

Answer. A story is a self-contained unit with a beginning, a middle and an end. If stories were concatenated into one long stream and cut into overlapping windows, the model would see sequences that begin mid-story or span two unrelated stories, would never learn that a story ends, and would be unable to learn where to begin or stop. Boundary preservation also keeps the data split clean: we split at the story level (10,000 / 1,000 / 1,000), so no part of a validation or test story appears in training and the test perplexity is not inflated by leakage. Each training item is one whole story between a beginning and an end marker.

Question. *Part 1: How is padding handled during training?*

Answer. Stories have different lengths (cut at 200 tokens), so each batch is padded to the longest story in it. The padding token is ignored in the cross-entropy loss, so padded positions do not contribute to the gradient or to perplexity, and the model is not rewarded for predicting padding.

Question. *Part 3: Do lower temperatures produce repetitive text?*

Answer. Yes. Greedy decoding (effectively temperature zero) is the worst case: trigram repetition is 0.772 (RNN) and 0.717 (LSTM), Distinct-2 is about 0.01, and the output is a loop ("it was a big, red car." repeated). A temperature of 0.5 still repeats (0.126 and 0.138). Repetition falls to 0.029 and 0.026 at temperature 0.8.

Question. *Part 3: Do higher temperatures lead to incoherent continuations?*

Answer. Yes. At temperature 1.2 repetition is essentially gone (0.002) and Distinct-2 is highest (0.619 and 0.644), but the text becomes ungrammatical and off-topic, with invented word combinations and unrelated characters.

Question. *Part 3: Does top-k sampling offer a useful compromise?*

Answer. Yes, in our experiments. With $k = 40$, repetition is 0.017 (RNN) and 0.016 (LSTM), lower than at temperature 0.8 (0.029 and 0.026), at a similar Distinct-2 (0.266 and 0.263 against 0.277 and 0.298). Because it removes the unlikely tail of the distribution, it avoids the garbled words of high temperatures without falling into greedy loops. Together with temperature 0.8 it gave the best balance.

Question. *Part 5: Does lower perplexity mean a better storyteller? Why might automatic and human judgement disagree?*

Answer. Not in our experiments. The RNN has the lower perplexity (25.13 versus 25.56 on test), yet the LSTM was rated more coherent (2.94 versus 2.78) and more relevant to the prompt (3.33

versus 3.06), while the RNN was rated slightly more grammatical and creative. The perplexity difference is only about 0.4, which is tiny, and the human differences are not reliable (raters disagreed by an average of 1.33 points on the same item, and each model had only six samples). More fundamentally, perplexity is measured under teacher forcing (the model always sees the correct previous words), but a generated story depends on the model's own earlier outputs, so small errors can accumulate (exposure bias). Perplexity also says nothing about plot, consistency or surprise, so lower perplexity is not expected to translate into better stories.

Question. *Part 6, Q1: How do the Vanilla RNN and the LSTM differ in modelling sequential text?*

Answer. In theory the LSTM's gated cell state lets gradients and information survive over many more steps than a Vanilla RNN's single repeatedly-transformed hidden state, which should help with long-range consistency. In our experiments the benefit was small: perplexity is nearly tied (RNN 25.13, LSTM 25.56). A likely reason is that TinyStories are short, written in simple children's language, with limited long-range dependencies, and each model was trained for only five epochs with one recurrent layer. The LSTM has 22% more parameters (6.51 M versus 5.33 M) without a perplexity advantage. We found some hints of better long-range behaviour, discussed next, but they are weak.

Question. *Part 6, Q2: Which model generates more coherent and less repetitive continuations?*

Answer. Slightly, the LSTM. Under greedy decoding it repeats less (0.717 versus 0.772), it stops on its own more often (93.5% of greedy outputs hit the length cap versus 99%, and 63% versus 83% at temperature 0.5), and it got higher coherence and relevance ratings. The evidence is weak: at temperature 0.5 the LSTM repeats a little more (0.138 versus 0.126), and the human evaluation is small. We therefore describe the LSTM as marginally better at staying on track and at ending a story, and not as clearly superior.

Question. *Part 6, Q3: How does temperature affect creativity and grammatical quality?*

Answer. Raising the temperature trades grammar for novelty. From 0.5 to 1.2, Distinct-2 rises from about 0.11 to about 0.62–0.64 and repetition falls from about 0.13 to 0.002, but samples go from repetitive but grammatical (0.5), to varied and mostly fluent (0.8), to ungrammatical and incoherent (1.2). The blinded human ratings covered only greedy and top-k samples, so the effect of temperature on ratings is based on our own reading of the samples and on the metrics, not on separate human scores.

Question. *Part 6, Q4: Can higher diversity scores guarantee better stories?*

Answer. No. The configuration with the highest diversity, temperature 1.2, produced the least readable text, and the most repetitive configuration, greedy, is the least diverse; neither extreme gave good stories. Diversity measures only how many different words and word pairs appear, and says nothing about whether they form a coherent plot. It should be read together with repetition rates and human judgement.

Question. *Part 6, Q5: Does lower perplexity correspond to better storytelling?*

Answer. No, as explained above. The lower-perplexity model (RNN) was not rated better overall, and the sampling method changed the output far more than the choice of model.

Question. *Part 6, Q6: What failures occur most frequently, and how might they be reduced?*

Answer. Loops and repetition (strongest in greedy decoding), topic drift (the opening about an exam paper or a message from a robot's future self was quickly abandoned for unrelated

characters), changing names, failure to stop (44%–99% of outputs hit the length cap, depending on setting), and at high temperature ungrammatical text. Possible remedies are a repetition or n-gram blocking penalty, nucleus (top-p) sampling, subword tokenisation to avoid unknown words and nonsense word forms, more training data and epochs, a larger or multi-layer model, and scheduled sampling or fine-tuning with sampled sequences to reduce exposure bias.

Question. *Part 6, Q7: What are the limitations of generating stories with a small language model trained on a limited corpus?*

Answer. The model has only a 6,414-word vocabulary, trained on 10,000 stories for five epochs, so its world knowledge, its sense of plot and its capacity to remember details across 150 tokens are all tiny. A plot twist needs a plan (set up an expectation, then violate it), and a next-word model with no explicit plan has no mechanism to do that, so it produces locally plausible sentences without global structure. In addition, the evaluation has limits: one training seed, twelve rated samples, three raters with large disagreement, and subjective rating criteria. Our conclusions are therefore indicative, not statistically conclusive.

8 Conclusion

Across the four questions, one lesson kept returning: giving a model better access to information usually helps, but it does not remove the need to look at *how* it fails. In machine translation, letting the decoder look at every encoder state tripled BLEU and improved every sentence-length bin, yet our attention maps also showed that confident-looking attention can still lead to the wrong word. In review rating, a pre-trained RoBERTa beat a GloVe BiGRU on accuracy, Macro-F1 and MAE even with a fifth of the data, and what we liked most is that its errors were milder: distant mistakes halved, so a wrong prediction was more often only one star away.

The summarization task taught us the most about how the pieces of a Transformer fit together: positional encoding to restore order, scaled dot-product attention, padding and causal masks, warm-up scheduling, and an autoregressive decoding loop. Implementing them from scratch makes it clear why a missing causal mask produces suspiciously good training loss and useless summaries, and why a small from-scratch model is expected to trail a simple Lead-3 baseline. We have documented that method in detail and will report the measured ROUGE scores and attention heatmap once the final run is committed.

In story generation, a plain RNN and an LSTM ended up nearly tied on perplexity, but how we decoded changed the output far more than which model we chose. Greedy decoding looped, very high temperature broke grammar, and the best stories came from moderate sampling. Throughout, small subsets, short training and few seeds mean the absolute numbers are modest; we compared models only under matched budgets, and we preferred to report limits openly rather than overstate results. Working as one team on equal terms made it easier to catch mistakes, and each of us now understands every one of these models rather than just one.

9 Repository and Reproducibility

All code, notebooks and outputs are at https://github.com/iamshashankyadav/NLP_GRA_2.

- `NLP_Assignment2_Q1_Attention_NMT.ipynb`: Q1 pipeline, models, heatmaps and analysis.
- `Q2_Roberta_vs_BiGRU_Amazon_Review.ipynb`: Q2 preprocessing, models, ablation and evaluation.

- `prepare.py, train.py, generate.py, evaluate.py, human_eval.py`: Q4 pipeline, run in that order.
- Q3 notebook: to be added to the repository.

Seeds: Q1 uses 42; Q2 uses 42 and 43; Q4 uses 42 for data and training and 0 and 1 for generation. Q1 and Q2 run top to bottom on a T4 GPU.

Contact

For any queries about this report or the code, please contact **Lakshay Gupta**: lakshayg@iitbhilai.ac.in, +91 99111 18016.